

COOLCITY: SUB-STREET CHILLED-WATER UTILITY SIMULATION

**Data Structures and Algorithms (C++) Mini Project
Report**

COVER PAGE

**CoolCity: Sub-Street Chilled-Water Utility
Simulation**

Data Structures and Algorithms (C++) Mini Project

Submitted By

Yatharth Dharmesh Mishra

Roll Number: 150096725055

Class: Larry Page

Submitted To

Dr. Aarti Pardeshi

College

ITM Skills University

Academic Year

2025–2029

1. STUDENT DETAILS

Field	Details
Student Name	Yatharth Dharmesh Mishra
Roll Number	150096725055
Class	Larry Page
Subject	Data Structures and Algorithms (C++)
Faculty Name	Dr. Aarti Pardeshi
Academic Year	2025–2029
Project Type	Individual Project
Project Title	CoolCity: Sub-Street Chilled-Water Utility Simulation

2. PROJECT TITLE

CoolCity: Sub-Street Chilled-Water Utility Simulation

CoolCity is a simulation-based system developed using C++ to model and manage a city's underground district cooling network. The project demonstrates the practical implementation of Data Structures and Algorithms in solving real-world infrastructure management problems.

3. PROBLEM STATEMENT

Modern district cooling systems transport chilled water through underground pipelines to provide cooling to multiple buildings from a centralized cooling plant. Managing such a network efficiently requires quick access to infrastructure data, reliable monitoring systems, fair request handling mechanisms, and intelligent resource allocation.

Traditional approaches face several challenges:

- Slow retrieval of pipe information.
- Lack of a rollback mechanism for dangerous pressure changes.
- Unorganized handling of building restart requests.
- Inefficient searching of pipes, valves, and pumps.
- No structured method to analyze cooling consumption.
- Absence of a complete underground utility network representation.
- Difficulty in finding the most energy-efficient pumping route.
- Inefficient power allocation among cooling fans.

The objective of this project is to design and implement a complete simulation system using Data Structures and Algorithms that addresses these challenges while improving efficiency, organization, and decision-making.

4. OBJECTIVES

The objectives of the CoolCity project are:

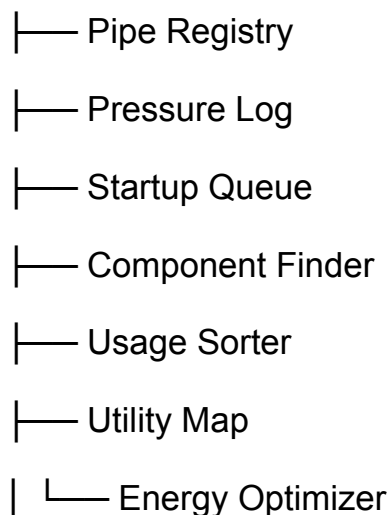
1. To maintain an organized registry of cooling pipes and specifications.
 2. To record pressure history and support rollback operations.
 3. To process cooling restart requests in a fair and sequential manner.
 4. To enable efficient searching of infrastructure components.
 5. To analyze and sort city blocks based on cooling consumption.
 6. To represent the underground cooling network using graph structures.
 7. To determine minimum-energy paths for chilled-water delivery.
 8. To allocate electrical power according to cooling demand priorities.
 9. To demonstrate practical applications of Data Structures and Algorithms.
 10. To improve understanding of STL containers and algorithm design in C++.
-

5. DESIGN AND ARCHITECTURE

The CoolCity system follows a modular architecture where each module addresses a specific operational challenge using an appropriate Data Structure or Algorithm.

System Architecture

CoolCity System



└─ Power Allocator

Module Description

Pipe Registry

Stores and manages all pipe specifications.

Pressure Log

Tracks pressure changes and enables rollback functionality.

Startup Queue

Handles building cooling restart requests in order.

Component Finder

Locates pipes, valves, and pumps efficiently.

Usage Sorter

Organizes city blocks according to cooling consumption.

Utility Map

Represents the underground network using graph structures.

Energy Optimizer

Calculates minimum-energy routes using shortest-path algorithms.

Power Allocator

Prioritizes electricity distribution based on cooling demand.

6. ALGORITHM DESCRIPTION

The project uses multiple Data Structures and Algorithms, each selected according to its suitability for the given task.

6.1 Pipe Registry

Data Structure:

`unordered_map<int, Pipe>`

Purpose:

Provides fast storage and retrieval of pipe information using hashing.

Operations:

- Add Pipe
 - Update Pipe
 - Delete Pipe
 - Search Pipe
 - Display Pipes
-

6.2 Pressure Log

Data Structure:

Stack

Purpose:

Stores pressure changes and allows rollback of the latest modifications.

Operations:

- Record Pressure
- View Current Pressure
- Undo Pressure Change

Reason:

Stack follows Last-In-First-Out (LIFO) behavior, making it ideal for undo operations.

6.3 Startup Queue

Data Structure:

Queue

Purpose:

Processes building restart requests in the exact order they are received.

Operations:

- Add Request
- Process Request
- View Pending Requests

Reason:

Queue follows First-In-First-Out (FIFO) behavior.

6.4 Component Finder

Data Structure:

Vector

Algorithms:

- Linear Search
- Binary Search

Purpose:

Find pipes, valves, and pumps using serial numbers.

Reason:

Linear Search works on unsorted data while Binary Search improves efficiency after sorting.

6.5 Usage Sorter

Data Structure:

Vector

Algorithm:

STL Sort

Purpose:

Sort city blocks based on cooling usage.

Features:

- Ascending Order
 - Descending Order
-

6.6 Utility Map

Data Structure:

Graph (Adjacency List)

Purpose:

Represents underground pipe connections and utility routes.

Features:

- Add Nodes
 - Add Connections
 - Display Network
-

6.7 Energy Optimizer

Algorithm:

Dijkstra's Algorithm

Purpose:

Find the minimum-energy path from the cooling plant to a building.

Reason:

Dijkstra efficiently calculates shortest paths in weighted graphs.

6.8 Power Allocator

Data Structure:

Priority Queue (Max Heap)

Purpose:

Allocate power according to cooling demand.

Reason:

Fans with higher power requirements receive priority.

7. COMPLEXITY ANALYSIS

Module	Data Structure / Algorithm	Time Complexity
Pipe Registry	Hash Table	$O(1)$ Average
Pressure Log	Stack	$O(1)$
Startup Queue	Queue	$O(1)$
Linear Search	Vector	$O(n)$
Binary Search	Sorted Vector	$O(\log n)$
Usage Sorting	STL Sort	$O(n \log n)$
Utility Map Traversal	Graph	$O(V + E)$
Dijkstra Algorithm	Priority Queue + Graph	$O((V + E) \log V)$
Power Allocation	Max Heap	$O(\log n)$

Where:

- V = Number of Vertices
- E = Number of Edges
- n = Number of Elements

8. SCREENSHOTS OF EXECUTION

The following screenshots demonstrate successful execution of the system:

Screenshot 1

Main Menu Interface

```
Explorer: C++-CASE-STUDY
├── .vscode
│   ├── launch.json
│   └── tasks.json
├── coolcity
├── CoolCity-SubStreet-ChilledWater-Utility-...
├── coolcity.cpp
└── DOCUMENTATION.md

Terminal: yatharth@yatharth-MacBook-Air C++-CASE-STUDY % ./coolcity
CoolCity: Sub-Street Chilled-Water Utility Simulation
===== MAIN MENU =====
1. Pipe Registry
2. Pressure Log
3. Startup Queue
4. Component Finder
5. Usage Sorter
6. Utility Map
7. Energy Optimizer
8. Power Allocator
9. Demo Simulation
10. Exit
Enter choice: 
```

Screenshot 2

Pipe Registry Operations

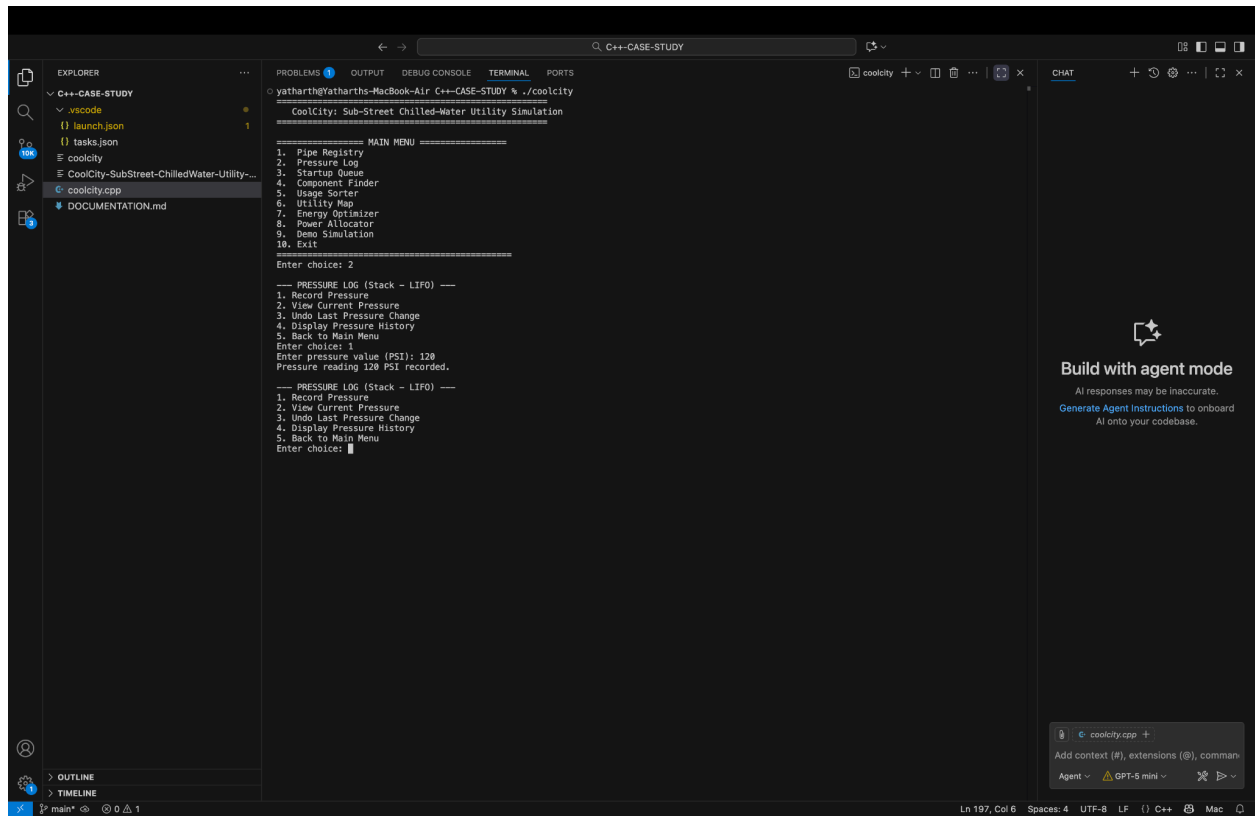
```
CoolCity: Sub-Street Chilled-Water Utility Simulation
=====
MAIN MENU =====
1. Pipe Registry
2. Pressure Log
3. Startup Queue
4. Component Finder
5. Usage Sorter
6. Utility Map
7. Energy Optimizer
8. Power Allocator
9. Demo Simulation
10. Exit
=====
Enter choice: 1

--- PIPE REGISTER (unordered_map / Hashing) ---
1. Add Pipe
2. Delete Pipe
3. Update Pipe
4. Search Pipe
5. Display All Pipes
6. Back to Main Menu
Enter choice: 1
Enter Serial Number: 1234
Enter Material: copper
Enter Diameter (in): 120
Enter Length (in): 180
Enter Pressure Rating (PSI): 120
Pipe 1234 added successfully.

--- PIPE REGISTER (unordered_map / Hashing) ---
1. Add Pipe
2. Delete Pipe
3. Update Pipe
4. Search Pipe
5. Display All Pipes
6. Back to Main Menu
Enter choice: 
```

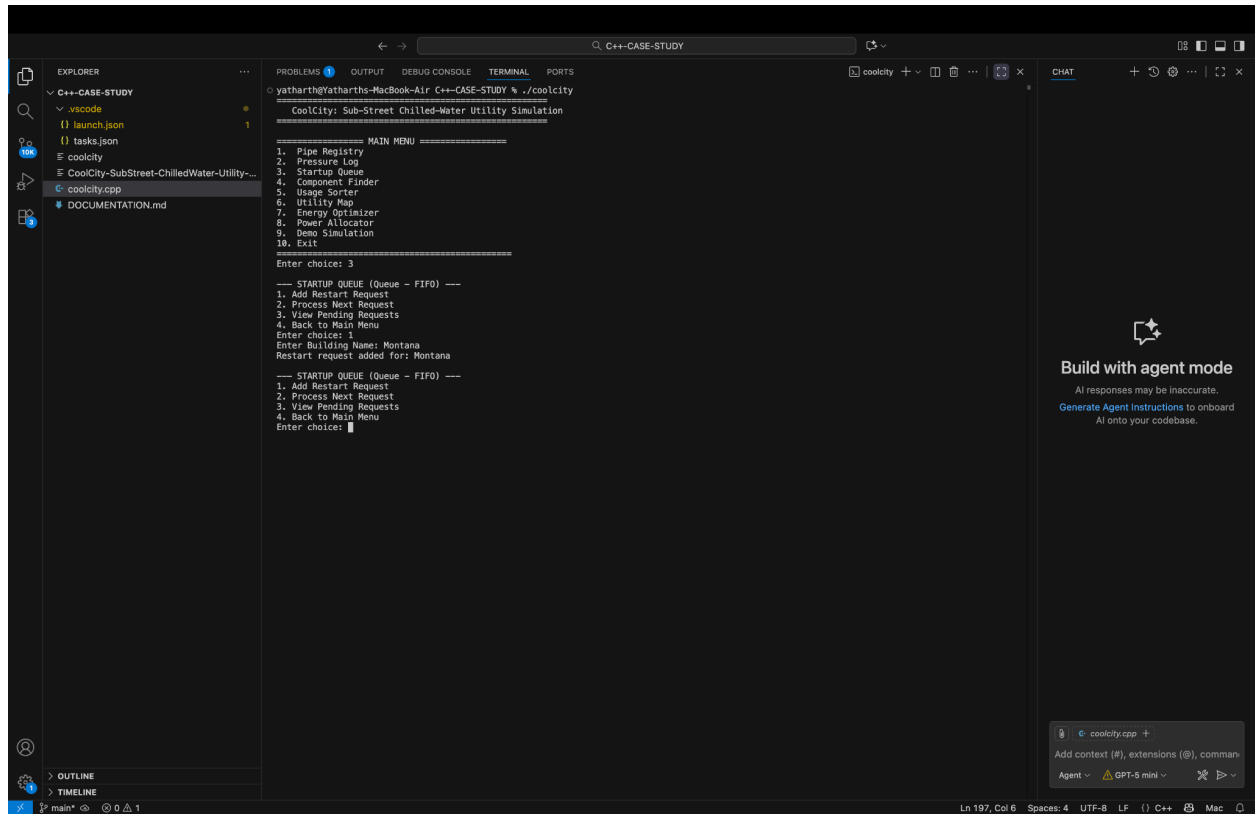
Screenshot 3

Pressure Logging and Rollback



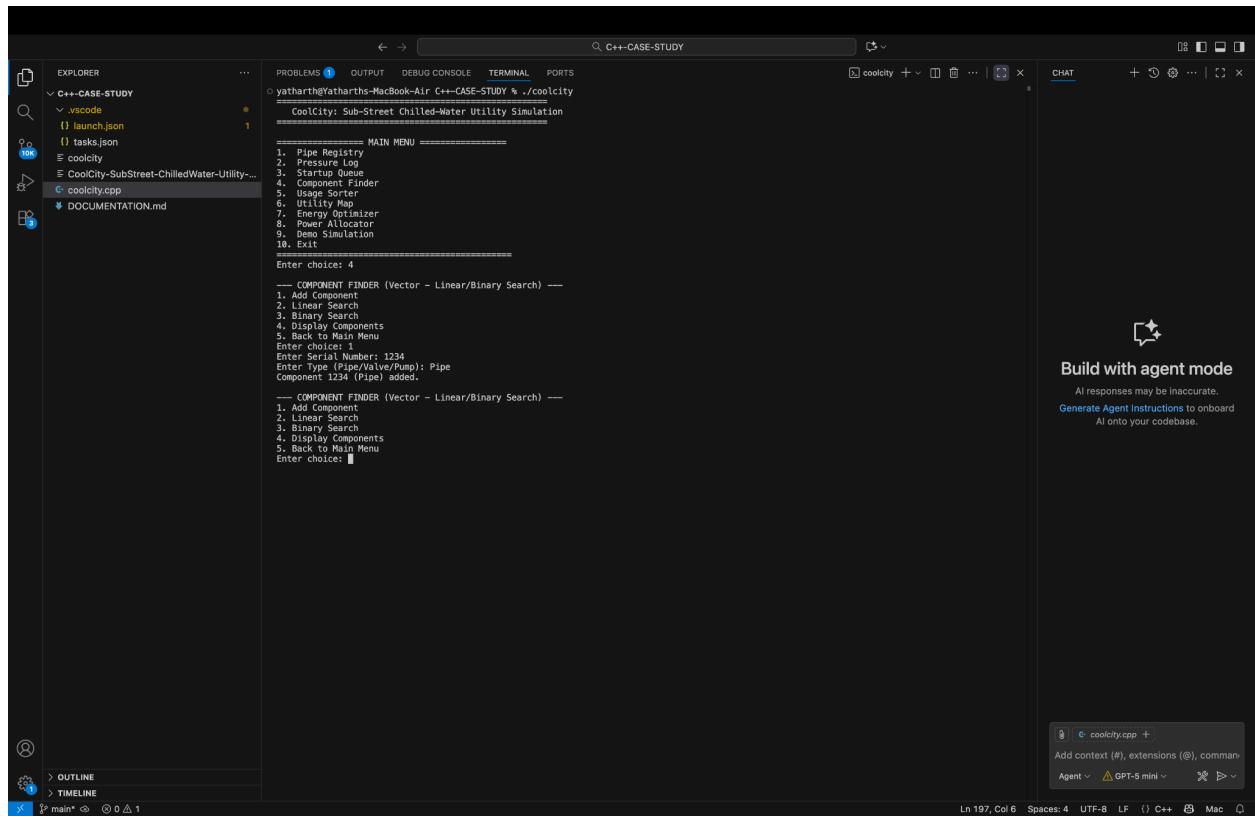
Screenshot 4

Startup Queue Processing



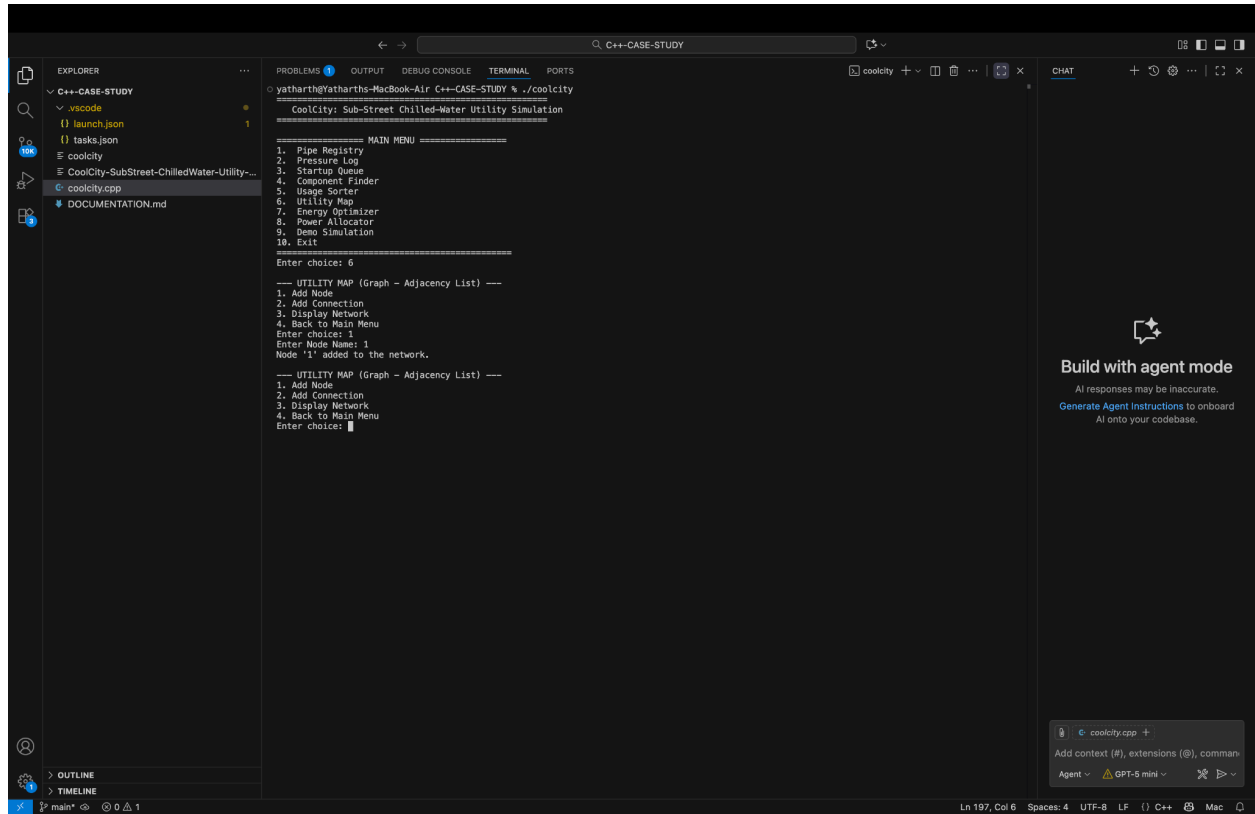
Screenshot 5

Component Search Functionality



Screenshot 6

Utility Map and Network Representation



Screenshot 7

Dijkstra Minimum-Energy Path Output

```

C++-CASE-STUDY
├── .vscode
│   ├── launch.json
│   └── tasks.json
├── coolcity
├── CoolCity-SubStreet-ChilledWater-Utility-...
├── coolcity.cpp
└── DOCUMENTATION.md

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
yatharth@yatharth-MacBook-Air C++-CASE-STUDY % ./coolcity
CoolCity: Sub-Street Chilled-Water Utility Simulation

===== MAIN MENU =====
1. Pipe Registry
2. Pressure Log
3. Startup Queue
4. Component Finder
5. Usage Sorter
6. Utility Map
7. Energy Optimizer
8. Power Allocator
9. Demo Simulation
10. Exit

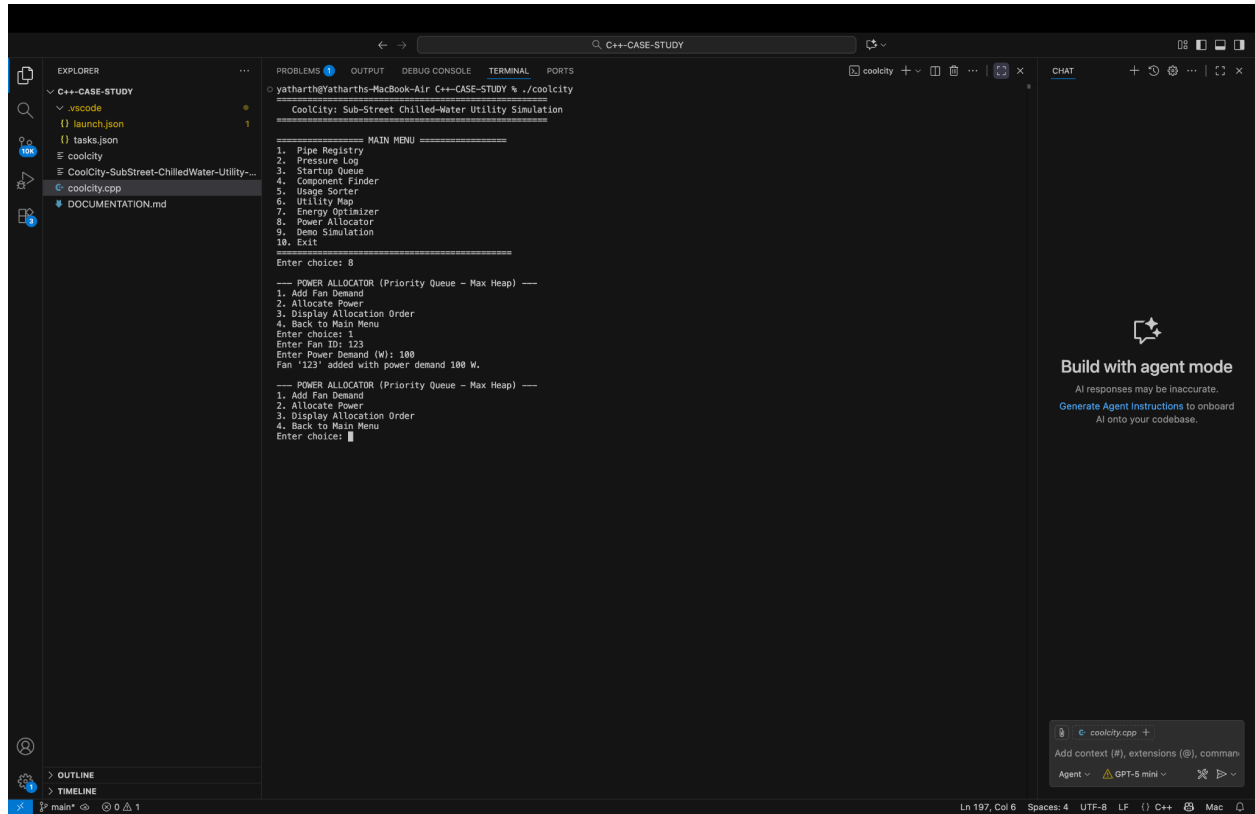
Enter choice: 7

--- ENERGY OPTIMIZER (Dijkstra's Algorithm) ---
1. Find Shortest Path & Total Energy Cost
2. Back to Main Menu
Enter choice: 1
Enter Source Node (e.g., Cooling Plant): Cooling Plant
Enter Destination Node (e.g., Building A): Error: One or both nodes do not exist in the network.

--- ENERGY OPTIMIZER (Dijkstra's Algorithm) ---
1. Find Shortest Path & Total Energy Cost
2. Back to Main Menu
Enter choice: 1
```

Screenshot 8

Power Allocation Using Priority Queue



```
Explorer: C++-CASE-STUDY
├── .vscode
│   ├── launch.json
│   └── tasks.json
├── coolcity
├── CoolCity-SubStreet-ChilledWater-Utility-...
├── coolcity.cpp
└── DOCUMENTATION.md

Problems: 0
Output: CoolCity: Sub-Street Chilled-Water Utility Simulation
CoolCity: Sub-Street Chilled-Water Utility Simulation
===== MAIN MENU =====
1. Pipe Registry
2. Pressure Log
3. Startup Queue
4. Component Finder
5. Usage Sorter
6. Utility Map
7. Energy Optimizer
8. Power Allocator
9. Demo Simulation
10. Exit
Enter choice: 8
===== POWER ALLOCATOR (Priority Queue - Max Heap) =====
1. Add Fan Demand
2. Allocate Power
3. Display Allocation Order
4. Back to Main Menu
Enter choice: 1
Enter Fan ID: 123
Enter Power Demand (W): 100
Fan '123' added with power demand 100 W.
===== POWER ALLOCATOR (Priority Queue - Max Heap) =====
1. Add Fan Demand
2. Allocate Power
3. Display Allocation Order
4. Back to Main Menu
Enter choice: 1
```

Build with agent mode
AI responses may be inaccurate.
Generate Agent Instructions to onboard
AI onto your codebase.

coolcity.cpp +
Add context (#), extensions (@), commands
Agent GPT-5 mini

9. RESULTS AND FINDINGS

The CoolCity simulation system successfully demonstrates the practical application of Data Structures and Algorithms in a real-world engineering scenario.

Key findings include:

- Hashing significantly improved pipe lookup efficiency.
- Stack-based rollback enabled quick recovery from pressure spikes.
- Queue-based processing ensured fair handling of restart requests.
- Binary Search reduced search time for sorted component data.
- Sorting algorithms facilitated better resource management.

- Graph structures effectively represented underground utility networks.
- Dijkstra's Algorithm successfully identified minimum-energy routes.
- Priority Queue improved power allocation efficiency.

The system achieved all functional objectives outlined in the project requirements.

10. CONCLUSION

The CoolCity: Sub-Street Chilled-Water Utility Simulation project successfully integrates multiple Data Structures and Algorithms into a single practical application.

The project demonstrates the implementation of:

- Hash Tables
- Stacks
- Queues
- Searching Algorithms
- Sorting Algorithms
- Graphs
- Dijkstra's Shortest Path Algorithm
- Priority Queues

Through this simulation, a realistic district cooling management system was developed that improves organization, efficiency, and resource allocation. The project highlights how Data Structures and Algorithms can be applied to solve real-world infrastructure challenges.

11. FUTURE SCOPE

The project can be further enhanced through:

- Graphical User Interface (GUI)
- Database Integration
- Real-Time Monitoring Systems
- IoT Sensor Connectivity
- Cloud-Based Data Storage
- Predictive Analytics for Energy Consumption
- Smart City Integration
- Mobile Dashboard Applications

These enhancements would make the system more scalable and suitable for real-world deployment.

12. GITHUB REPOSITORY LINK

Repository Name:

coolcity-dsa-simulation

GitHub Repository:

<https://github.com/2025yatharthm-gif/coolcity-dsa-simulation>

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Dr. Aarti Pardeshi for her guidance, support, and valuable suggestions throughout the development of this project. I also thank ITM Skills University for providing the opportunity and resources necessary to complete this project successfully.

End of Report